

DoSync: A Semantic Layer for IoT Device Orchestration by AI Agents

Rodrigo Giuliani
Independent Researcher
Córdoba, Argentina
giulianireg@gmail.com

Abstract—Existing IoT protocols assume a human decides what to do and a device executes it. This model fails when AI agents — which express semantic goals rather than specific commands — are introduced. DoSync is an open-source communication protocol (Apache 2.0) that introduces a semantic layer between AI agents and physical devices. Unlike approaches that require LLMs in the critical execution path [1] or non-deterministic goal-oriented reasoning [2], DoSync adopts a declarative model: each device publishes a Capability Manifest declaring its capabilities and context, and a deterministic resolver constructs the action plan at runtime without pre-written rules. The protocol was evaluated on a reference implementation running on a Raspberry Pi 5 with 38 real physical devices. The resolver operates under 0.11 ms p99 and scales to 5,000 devices within the 500 ms specification limit. A semantic accuracy evaluation over 15 real scenarios yields mean precision of 0.85 and recall of 0.49, with $F1=1.00$ on safety-critical intents. Two independent implementations — Python and Node.js — validate the specification as an interoperable protocol. Results should be interpreted as an architectural feasibility validation rather than a definitive validation of the semantic model.

Index Terms—IoT, AI agents, semantic protocol, orchestration, intent, MCP, capability manifest

1. Introduction

The proliferation of IoT devices and the emergence of capable AI agents have created an architectural gap that existing protocols do not address. Protocols such as Matter [3], Zigbee [4], and Z-Wave [5] were designed for the human-in-the-loop model: a user decides what to do, an application translates that decision into a command, and the device executes it.

The problem emerges when the human is replaced by an AI agent. An agent detecting a fall through computer vision does not produce `phone.call("911")` — it produces an understanding of the situation: *"there is an emergency."* The translation from that understanding to specific device commands must be written manually for every possible scenario, breaks when a new device is added, and scales unsustainably with environment complexity. We identify this as the *command gap*.

Recent work addresses this problem in different ways. LLMind [1] uses LLMs as orchestrators that generate FSM-based control scripts — powerful but introduces non-determinism in the critical execution path. SASHA [2] applies goal-oriented reasoning with LLMs directly on home devices — flexible but neither deterministic nor auditable. DoSync takes a different approach: rather than having the agent dynamically generate how to act, devices declare their capabilities and the resolver determines the action plan deterministically.

Primary scientific contribution. This work demonstrates that a declarative capability-based architecture can resolve the gap between AI agent semantic intentions and physical devices without requiring non-deterministic reasoning or LLMs in the critical execution path. The central hypothesis is that knowledge of *how to respond* to an intention can be distributed to the devices themselves via capability manifests, enabling automatic discovery and deterministic resolution. This separation — agent reasons over goals, infrastructure executes predictably — is the principal architectural contribution.

Specific contributions:

- 1) Formalization of the command gap in IoT systems with AI agents
- 2) A declarative Capability Manifest model for automatic device discovery
- 3) A deterministic semantic resolver benchmarked on real production hardware
- 4) Semantic precision and recall evaluation over 15 real scenarios (H3)
- 5) A configurable policy engine with 5 policy types for critical environments
- 6) A tamper-evident audit log via SHA-256 chaining [12], [15]
- 7) A Device Health Monitor with real production data
- 8) Two independent implementations validating the specification

2. Related Work

Table 1 compares DoSync with related systems across five dimensions.

The fundamental difference from Matter, Home Assistant, and openHAB is *automatic discovery*: adding a device

TABLE 1. COMPARISON WITH RELATED SYSTEMS

System	Model	Disc.	Det.	Audit	LLM
Matter [3]	Cmnds	Manual	Yes	No	No
Home Asst. [6]	Rules	Manual	Yes	Partial*	Opt
openHAB [7]	Items	Manual	Yes	No	No
LLMind [1]	FSM	Auto	No	No	Yes
SASHA [2]	LLM	Auto	No	No	Yes
DoSync	Intents	Auto	Yes	SHA-256	No

*HA logs history without cryptographic chaining.

Disc. = Auto-discovery; Det. = Determinism; LLM = LLM in critical path.

requires no rule or configuration changes. The fundamental difference from LLMind and SASHA is *determinism*: both require an LLM in the critical execution path, introducing response variability.

Semantic interoperability in IoT has been addressed through formal ontologies such as SOSA/SSN [8] and W3C WoT [9]. DoSync differs in requiring no RDF graph reasoning — the Capability Manifest is a flat JSON structure any device can publish without external dependencies. CASIT [11] proposes an LLM-based multi-agent IoT system; unlike CASIT, DoSync delegates LLM reasoning to the external agent and keeps the hub deterministic.

3. System Design

3.1. Five-Layer Architecture

DoSync organizes communication in five layers: (5) *Intent* — AI agent expresses semantic goals; (4) *Semantic* — resolver maps intent $\rightarrow$ ActionPlan; (3) *Registry* — devices declare capabilities via Capability Manifest; (2) *Security* — TLS 1.3 with local PKI, no internet required; (1) *Transport (HAL)* — abstraction over WiFi, BLE, Zigbee, Z-Wave, and Thread.

3.2. Capability Manifest and Intent Classes

Each device publishes a manifest declaring tags, actuators, sensors, and `emergency_capable`. Tags are the primary semantic resolution mechanism; `emergency_capable` guarantees inclusion in emergency scenarios regardless of computed score.

The protocol defines 13 intent classes by priority: **Safety** (`ensure_safety`, `alert_anomaly`), **Access** (`control_access`), **Presence** (`children_arrived_home`, `notify_family`), **Comfort** (`set_environment`, `morning_routine`, `bedtime_routine`), **Efficiency** (`save_energy`, `away_mode`), and **Monitoring** (`monitor_health`, `report_status`, `remind_chore`).

3.3. Semantic Resolver

H1: A capability-based scoring algorithm can select relevant devices with latency under 500 ms for registries up to 5,000 devices.

H2: Semantic resolution overhead represents less than 1% of total execution time in real deployments.

The `CapabilityMatchingResolver` computes a relevance score for each registered device:

$$s = t \times 10 + l \times 15 + e \times 30 + a \times 8 \quad (1)$$

where t = tag overlap count, l = location match (0/1), e = emergency bonus (0/1), a = actuator match count. The `emergency_bonus` weight (30) is triple `tag_overlap` (10) because a false negative in an emergency has worse consequences than a false positive. The resolver is **deterministic**: same intent and registry always produce the same ActionPlan.

The `StateAwareResolver` extends this by filtering redundant actions, persisting device state in SQLite across restarts.

3.4. Policy Engine and Audit Log

The policy engine evaluates the ActionPlan before execution with five types: `NeverAfterHoursPolicy`, `RequireConfirmationPolicy`, `BlockIntentPolicy`, `DeviceExclusionPolicy`, and `ConflictResolutionPolicy`. Emergency intents bypass all policies except security ones.

Each audit entry is SHA-256 chained: $h_n = \text{SHA256}(e_n \parallel h_{n-1})$

Modifying any entry invalidates all subsequent hashes, verified in real time [12], [15].

3.5. Device Health Monitor

Records the outcome of each adapter execution in SQLite. Exposes `GET /v1/health/devices` with per-device statistics and configurable alert thresholds. Follows the principle of *observability without autonomy*: the system alerts, but decisions rest with the human operator.

4. Implementation

The reference hub uses FastAPI with 14 REST endpoints and a real-time WebSocket. SQLite persists 6 tables. It runs on Raspberry Pi 5 as a systemd service with TLS 1.3. Available adapters: `WiZAdapter` (UDP, Philips WiZ), `HABridge` (Home Assistant, 10 domains), `NotificationAdapter` (SMS via Twilio), `GPIOAdapter` (PIR + DHT22).

A native MCP server [13] exposes the hub as a toolset for any compatible AI agent (Claude, ChatGPT, Hermes Agent) without additional integration code.

An independent Node.js implementation — no shared code with Python — passes the certification suite Basic

tier (6/6 tests), validating that the specification is implementable by third parties. The certification suite includes 16 tests across 3 tiers: Basic (connectivity, registration, manifests), Standard (intents, events, error validation), Emergency (emergency override, SHA-256 integrity).

5. Evaluation

5.1. Experimental Setup

Registry: 38 real devices from the production hub (Raspberry Pi 5, Arm Cortex-A76, 8 GB RAM). **Iterations:** 500 per resolver. **Seed:** 42 (reproducible). **Intents:** randomly sampled across 13 classes, 3 urgency levels, 5 location contexts.

5.2. H1 — Latency and Scalability

TABLE 2. RESOLVER LATENCY VS. REGISTRY SIZE

Devices	Mean	p95	p99	≤500 ms
38 (prod.)	0.053 ms	0.074 ms	0.107 ms	✓
100	0.096 ms	0.141 ms	0.196 ms	✓
500	0.498 ms	0.737 ms	1.486 ms	✓
1,000	1.013 ms	1.375 ms	3.044 ms	✓
5,000	5.300 ms	9.129 ms	11.392 ms	✓

H1 confirmed. The resolver operates within the 500 ms limit up to 5,000 devices. The algorithm is $O(n)$ — at 5,000+ devices p95 exceeds 9 ms; tag indexing to achieve $O(1)$ is planned for v0.3. The `StateAwareResolver` eliminates **35% of redundant actions** with equivalent latency (p99: 0.109 ms vs 0.107 ms).

5.3. H2 — Semantic Overhead

The semantic resolver adds 0.051 ms of absolute overhead over a direct command. In real deployment context (WiFi → WiZ: 5–15 ms; WiFi → HA: 20–80 ms), the semantic layer represents **less than 1%** of total execution time. **H2 confirmed.**

5.4. H3 — Semantic Resolution Accuracy

H3: The resolver selects relevant devices with precision above 0.80.

A manual ground truth was defined for 15 representative scenarios over the 38-device registry. Expected devices were defined by the author based on expert knowledge of the production deployment; evaluation by independent raters is planned as future work.

H3 partially confirmed. Mean precision is 0.85 — the resolver rarely includes incorrect devices. Mean recall of 0.49 indicates that in several scenarios not all relevant devices are selected. Analysis reveals a consistent pattern: intents with broad resolution tags (`save_energy`,

TABLE 3. SEMANTIC RESOLUTION ACCURACY (15 SCENARIOS)

Scenario	Prec.	Rec.	F1
General emergency	1.00	1.00	1.00
Security anomaly	1.00	1.00	1.00
Children arrived	1.00	1.00	1.00
Notify family	1.00	0.33	0.50
Save energy	1.00	0.08	0.15
Bedtime routine	0.00	0.00	0.00
Morning routine	0.79	1.00	0.88
Away mode	1.00	0.08	0.15
Set environment	1.00	0.00	0.00
Alert anomaly	1.00	0.50	0.67
Report status	1.00	1.00	1.00
Monitor health	1.00	0.00	0.00
Remind chore	1.00	0.33	0.50
Access control	0.00	1.00	0.00
Environment + location	1.00	0.00	0.00
Average	0.85	0.49	0.46

`away_mode`, `bedtime_routine`) produce low recall because the registry lacks devices with specific domain tags (`thermostat`, `blinds`). Safety-critical and communication intents achieve F1=1.00, confirming correct behavior for highest-priority scenarios.

5.5. Device Health Monitor — Production Data

TABLE 4. DEVICE HEALTH MONITOR — PRODUCTION (31 EXECUTIONS, 7 DEVICES)

Device	OK	Total	Rate
notifier-sms-01	6	6	100%
wiz-habitacion-principal	1	1	100%
wiz-habitacion-ninos-01	0	12	0%
wiz-living1-01	0	12	0%
wiz-living1-02	0	12	0%
wiz-living2-01	0	12	0%
wiz-living2-02	0	12	0%

Note: data reflects early accumulation period; patterns are preliminary.

The 5 WiZ lights with 0% success correspond to `save_energy` executions during nighttime with lights physically off — the adapter sent UDP commands but devices did not respond in low-power state. This illustrates a documented `StateAwareResolver` limitation: on cache miss, the resolver includes the action even if potentially redundant. Direct device state querying before resolution is planned for v0.4.

5.6. Audit Log Integrity

474 entries accumulated in production. SHA-256 chain intact across all queries since deployment.

6. Limitations and Future Work

Empirical scoring weights. Weights were defined over production scenarios without formal derivation.

Low recall on comfort/efficiency intents. Mean recall of 0.49 is driven by generic tag configurations in manifests. Deployment guidelines to maximize recall are priority future work.

Non-distributed state. State is not consistent across multi-hub deployments.

Partial failure model. No compensation or rollback when an adapter fails mid-ActionPlan.

Single ground truth evaluator. Independent rater validation is planned.

Conceptual comparison with existing systems. Table 1 is qualitative; quantitative comparison (rules count, onboarding time) is planned.

Planned: v0.3 (tag indexing, O(1) lookup), v0.4 (distributed state, direct device state querying), v1.0 (stable interface, formal governance).

7. Conclusion

DoSync demonstrates that a declarative capability-based architecture can introduce a semantic layer between AI agents and IoT devices while preserving determinism, auditability, and security — properties that LLM-based critical-path approaches do not guarantee. The Capability Manifest model enables automatic device discovery without manual configuration.

Empirical results confirm the stated hypotheses: the resolver operates within specification limits up to 5,000 devices (H1), semantic overhead is under 1% of total execution time (H2), and mean precision reaches 0.85 with F1 = 1.00 on safety-critical intents (H3 partially confirmed). Production Device Health Monitor data reveals a concrete cache-miss failure pattern that directly informs the protocol roadmap.

Two independent implementations in different languages, with no shared code, validate that the specification is precise enough for third-party implementation — the minimum criterion for an open standard.

Results should be interpreted as an architectural feasibility validation rather than a definitive validation of the semantic model. The evaluated deployment is a domestic environment with 38 real devices; generalization to industrial or larger-scale environments requires additional independent evaluation.

Code, specification, and certification suite: <https://github.com/giulianireg-spec/dosync-protocol> (Apache 2.0).

References

- [1] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, “LLMind: Orchestrating AI and IoT with LLM for Complex Task Execution,” *IEEE Internet of Things Journal*, 2024. DOI: 10.1109/JIOT.2024.10697418
- [2] E. King, H. Yu, S. Lee, and C. Julien, “Sasha: Creative Goal-Oriented Reasoning in Smart Homes with Large Language Models,” *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 8, no. 1, 2024.
- [3] Connectivity Standards Alliance, “Matter Specification v1.3,” 2024. [Online]. Available: <https://csa-iot.org>
- [4] Zigbee Alliance, “Zigbee Specification R21,” 2015.
- [5] Z-Wave Alliance, “Z-Wave Specification,” 2022.
- [6] Home Assistant, “Architecture Overview,” <https://www.home-assistant.io/docs/architecture/>, 2024.
- [7] openHAB Community, “openHAB Developer Documentation,” <https://www.openhab.org/docs/>, 2024.
- [8] A. Haller *et al.*, “The modular SSN ontology: A joint W3C and OGC standard,” *Semantic Web*, vol. 10, no. 1, pp. 9–32, 2019. DOI: 10.3233/SW-180320
- [9] W3C, “Web of Things (WoT) Thing Description,” W3C Recommendation, 2020. Available: <https://www.w3.org/TR/wot-thing-description/>
- [10] H. Moeini, I.-L. Yen, and F. Bastani, “Summarization in Semantic Based Service Discovery in Dynamic IoT-Edge Networks,” arXiv:2009.02858, 2020.
- [11] N. Zhong *et al.*, “CASIT: Collective Intelligent Agent System for Internet of Things,” *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 19646–19656, 2024. DOI: 10.1109/JIOT.2024.3366906
- [12] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. Available: <https://bitcoin.org/bitcoin.pdf>
- [13] Anthropic, “Model Context Protocol Specification,” <https://modelcontextprotocol.io/>, 2024.
- [14] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [15] E.-O. Blass and G. Noubir, “Accountability of Things: Large-Scale Tamper-Evident Logging for Smart Devices,” arXiv:2308.05557, 2023.